

Rock Paper Scissors – Python Course Project

1. Project Overview

Rock Paper Scissors is a simple command-line game made using Python. In this game, the player chooses one option from **rock**, **paper**, or **scissor**, and the computer randomly chooses one of the same three options.

After both choices are made, the program compares them and displays the result. The player can either win, lose, or get a tie. The program also checks invalid input and gives the player an option to play another round.

This project was made to practice basic Python concepts such as tuples, loops, user input, the `random` module, string methods, and `if`, `elif`, and `else` conditions.

2. Features

The project includes the following features:

- Random selection by the computer
- Player input through the terminal
- Lowercase conversion of user input
- Invalid input checking
- Win, lose, and tie conditions
- Option to play another round
- Simple command-line interface
- No external packages required

3. Technologies Used

Programming Language: Python 3

Python Module Used: `random`

Execution: Command Prompt / Terminal

4. Project Structure

Rock-Paper-Scissors/

rock_paper_scissors.py
README.md
Project_Report.pdf

5. Requirements

Python 3 is required to run this project.

No additional packages or libraries need to be installed because the project only uses the built-in `random` module.

To check whether Python is installed, open the terminal and enter:

```
python --version
```

On some systems, the following command can be used:

```
python3 --version
```

6. How to Run the Project

Open the terminal in the project folder.

Run the following command:

```
python rock_paper_scissors.py
```

If your system uses `python3`, use:

```
python3 rock_paper_scissors.py
```

After starting the program, it will ask the player to enter a choice.

Example:

```
Enter a choice(Rock,Paper,scissor): rock
Player : rock
Computer : scissor
You won !
Play again ? (y/n)y
```

7. How the Program Works

The program works in a simple sequence.

First, the three available choices are stored in a tuple. Then the computer selects one of them randomly using the `random.choice()` function.

The player is asked to enter a choice. The entered value is converted into lowercase using the `.lower()` string method so that inputs such as `Rock` and `rock` are treated in the same way.

The program then checks whether the entered value is one of the valid choices. If the input is invalid, the program displays **Invalid Input** and asks the player to enter a choice again.

When the input is valid, the program compares the player's choice with the computer's choice.

If both choices are the same, the result is a tie.

The program then checks the three possible winning combinations for the player:

Rock defeats Scissor.
Paper defeats Rock.
Scissor defeats Paper.

If none of the winning conditions is true, the player loses.

After showing the result, the program asks the player whether they want to play another round.

8. Rules of the Game

Player Choice	Computer Choice	Result
Rock	Scissor	Player Wins
Paper	Rock	Player Wins
Scissor	Paper	Player Wins
Same Choice	Same Choice	Tie
Other Valid Combination	—	Player Loses

9. Input Validation

The program accepts only the following three choices:

rock
paper
scissor

If the player enters an unsupported value, the program displays:

Invalid Input

and asks for the choice again.

For example:

Enter a choice(Rock,Paper,scissor): hello

Invalid Input

The player can then enter a valid choice.

10. Testing

The program was tested using different inputs and game situations.

The following cases were checked:

1. Valid input such as **rock**, **paper**, and **scissor**.
2. Invalid input such as **hello**.
3. Tie condition when both choices are the same.
4. Rock winning against scissor.
5. Paper winning against rock.
6. Scissor winning against paper.
7. Losing conditions.
8. Playing another round.
9. Ending the game when the player does not choose **y**.

These tests help check whether the main game logic and input validation are working correctly.

11. Limitations

The current version is a basic command-line game.

It does not currently include:

- Score tracking
- Player accounts
- Match history
- Graphical user interface
- Different game modes
- Saved game data

12. Future Improvements

The project can be improved in the future by adding score tracking so that the number of wins and losses can be displayed.

Other possible improvements include adding multiple rounds, a graphical user interface, saving match history, and adding different game modes.

Conclusion

This project helped in understanding how different Python concepts can be combined to create a small working application. It provided practice with loops, tuples, user input, conditions, string methods, and the `random` module.

The project is simple, but it gives a practical example of how basic Python concepts can be used together to build an interactive command-line program.